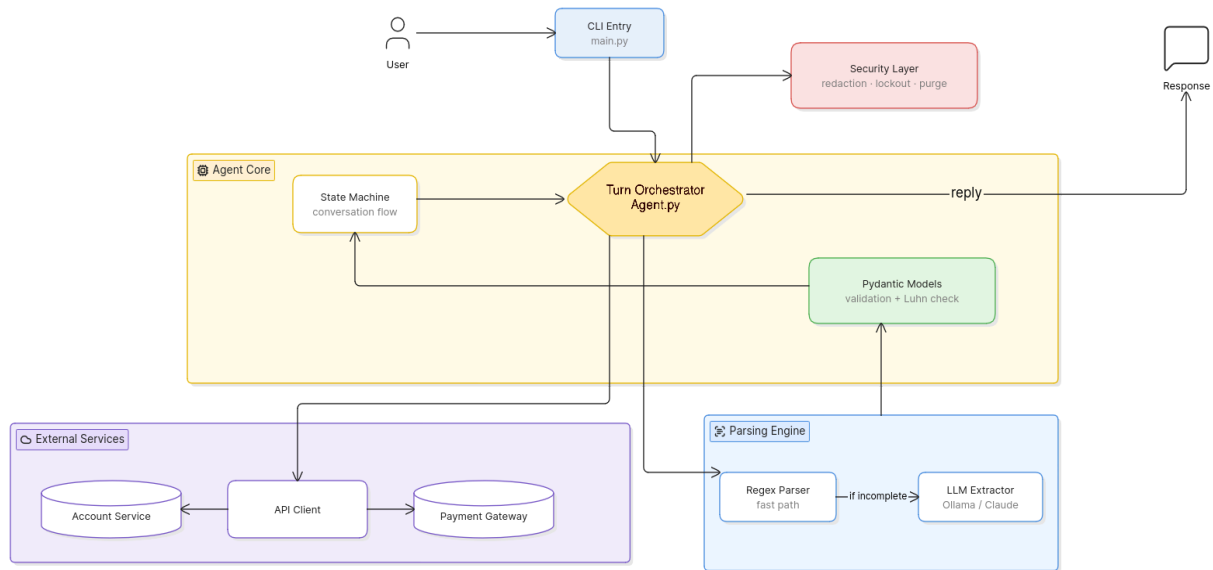


Architecture:



Architecture Overview

- **Turn-Oriented Architecture:** The agent is driven by a central Turn Orchestrator (`Agent.next()` in `agent.py`), which processes one user message at a time and coordinates all components.
- **Finite State Machine (FSM):** Conversation flow is managed using a state machine, ensuring deterministic transitions between identity verification, payment collection, processing, and completion.
- **Hybrid Parsing Engine:** User inputs are first processed by a regex-based parser for fast extraction of structured data. If information is ambiguous or incomplete, an LLM extractor is used as a fallback.
- **Data Validation Layer:** All extracted information is validated using Pydantic models, including format validation (e.g., Aadhaar Luhn check) before being accepted into the conversation state.
- **External Service Integration:** A dedicated API client handles communication with the Account Service (for account lookup) and Payment Gateway (for payment processing), keeping external interactions isolated from business logic.
- **Security Layer:** A centralized security module enforces sensitive data redaction, verification lockout policies, and temporary data cleanup after the conversation ends.
- **Modular Design:** Parsing, orchestration, validation, security, and API communication are implemented as separate components, improving maintainability, testability, and extensibility.

Key Decisions Made:

1. Use an explicit finite-state machine

- The flow moves through fixed stages: account lookup → identity verification → balance → amount → card details → payment.

Why: Prevents payment before verification, makes behavior predictable, and simplifies testing.

2. Use deterministic parsing first, with optional LLM extraction

- Regex and normalization run on every turn. An LLM extractor is used only when required information remains unclear, and only for a specific schema such as identity, amount, or card data.

Why: Natural language is supported without allowing probabilistic model output to make payment or verification decisions.

3. Require strict identity verification with logout

- Verification requires an exact normalized full-name match plus one valid secondary factor: DOB, Aadhaar last four digits, or pincode. Three complete failures close the conversation.

Why: Prevents account enumeration and makes repeated guessing difficult.

4. Validate payment data locally before calling the API

- Amounts use Decimal, balances are checked locally, and card numbers undergo normalization, length checks, Luhn validation, expiry validation, and CVV validation. API clients are injectable for testing.

Why: Avoids unnecessary or unsafe API calls and makes the behavior testable without live services.

Tradeoff's Accepted:

1. Deterministic state machine over fully conversational flexibility - safer validation, predictable behavior, and easier testing, but less natural than an end-to-end LLM agent.
2. Regex/parsing first, LLM extraction second - improves reliability and cost control, but some natural-language inputs may be rejected or require fallback extraction.
3. Strict exact-name and verification matching - reduces account-takeover risk, but can reject legitimate users with nicknames, aliases, spelling differences, or outdated records.
4. Clearing all card data after every payment attempt - reduces sensitive-data retention, but forces users to re-enter valid card details after recoverable failures.
5. Synchronous API calls - keeps state transitions simple and deterministic, but increases conversation latency and limits throughput.
6. In-memory conversation state only - keeps the implementation simple and reduces persistence risk, but conversations cannot resume after process restarts or move between workers.

Improvements:

1. **Use tokenized payments instead of collecting raw card data** - Integrate a PCI-compliant payment provider or hosted payment form. Card numbers and CVVs should never pass through the agent or an LLM.
2. **Add persistent sessions and recovery** - Store conversation state securely so users can resume after a restart or network failure, while preventing duplicate payments with idempotency keys.
3. **Improve production reliability and observability** - Add structured redacted logs, metrics, tracing, rate limiting, circuit breakers, and clearer monitoring for API failures.
4. **Make extraction and conversation handling more flexible** - Support multilingual input, stronger ambiguity detection, confidence-based clarification, accessibility improvements, and human-agent handoff.
5. **Expand testing and deployment automation** - Add property-based and fuzz testing for parsers, API contract tests, CI pipelines, security scanning, containerization, and staging-environment smoke tests.